Department of Electrical Engineering

Indian Institute of Technology, Kharagpur

Algorithms, AI and ML Laboratory (EE22202)

Spring, 2025-26

Report 4: Decision Trees

Name: Arijit Dey

Roll No: 24IE10001

Decision Trees

This report outlines the implementation of the standard ID3 Decision Tree Algorithm and measuring its performance while tuning the model and the dataset with different scenarios. For the dataset, we use the Car Evaluation Database which was originally designed for demonstration of the hierarchical decision model.

1 Loaing the Dataset

The dataset is loaded using pandas. The columns are named according to the `car.names` file. The last column, "accept", is the target variable, while the others are attributes.

Loading Dataset with Pandas

```python
df = pd.read_csv("car.data", header=None, names=cols)

attributes = list(df.columns[:-1])
target = df.columns[-1]
X = df[attributes]
y = df[target]
```

Result

```
Attributes:
  buying  maint doors persons lug_boot safety
0  vhigh  vhigh     2       2    small    low
1  vhigh  vhigh     2       2    small    med
2  vhigh  vhigh     2       2    small   high
3  vhigh  vhigh     2       2      med    low
4  vhigh  vhigh     2       2      med    med

Target:
0    unacc
1    unacc
2    unacc
3    unacc
4    unacc
Name: accept, dtype: str
```

2 ID3 Decision Tree Implementation

The `ID3DecisionTree` class encapsulates the logic for building the tree, making predictions, and evaluating performance. Key methods include impurity calculation (Entropy, Gini), information gain, and the recursive `fit` method to construct the tree.

ID3DecisionTree Class Structure

```python
class ID3DecisionTree:
    def __init__(self, impurity_metric='entropy', max_depth=None, max_impurity=0.0):
        # -- SETUP CODE --
```

2.1 Impurity Calculation

When the `impurity_metric` is `entropy`, we calculate the impurity as follow:

$$H(S) = -\sum_{i=1}^{N} p_i \log_2(p_i)$$

whereas when `impurity_metric` is `gini`, the calculation is done as:

$$H(S) - \sum_{i=1}^{N} p_i^2$$

In both cases, N is the count of the unique classes in that particular column and p_i is probablity of obtaining the class S from the column.

Total Impurity

```python
def _get_impurity(self, y):
    if len(y) == 0: return 0
    probs = y.value_counts(normalize=True)
    if self.impurity_metric == 'entropy':
        return -np.sum(probs * np.log2(probs + 1e-9))
    elif self.impurity_metric == 'jgini':
        return 1 - np.sum(probs**2)
```

2.2 Information Gain

The information gain is computed by:

$$IG(S, A) = H(S) - \sum_{v \in \text{ Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

Information Gain

```python
def _information_gain(self, X, y, attribute):
    total_impurity = self._get_impurity(y)
    values = X[attribute].unique()
    weighted_impurity = 0
    split_info = 0
    for val in values:
        subset_y = y[X[attribute] == val]
        weight = len(subset_y) / len(y)
        weighted_impurity += weight * self._get_impurity(subset_y)
        split_info -= weight * np.log2(weight + 1e-9)
    gain = total_impurity - weighted_impurity
    if self.impurity_metric == 'gain_ratio':
        return gain / (split_info + 1e-9)
    return gain
```

2.3 Training the Model

1. At each level of the recursion, we calculate the information gain for each of the attributes and pick the attribute with the highest gain. Then we loop through all the unique values of this attribute.
2. For each value, we filter in only the dataset rows where the this value is present and remove the rest.
3. Then we recursively call this function to furthur build up the tree.
4. In case we exceed the maximum allowed depth, we stop with leaf node to be the most frequently occuring target value in that subset of rows.

```
Recursive training function
    def fit(self, X, y, depth=0):
        # ... PERFORMANCE TRACKING CODE
        if len(y.unique()) == 1 or (self.max_depth and depth >= self.max_depth):
            return y.mode()[0]

        gains = {attr: self._information_gain(X, y, attr) for attr in X.columns}
        if not gains or max(gains.values()) <= 0:
            return y.mode()[0]

        best_attr = max(gains, key=gains.get)
        current_node_tree = {best_attr: {}}

        for val in X[best_attr].unique():
            X_subset = X[X[best_attr] == val].drop(columns=[best_attr])
            y_subset = y[X[best_attr] == val]
            if not y_subset.empty:
                current_node_tree[best_attr][val] = self.fit(X_subset, y_subset, depth + 1)

        if depth == 0:
            self.tree = current_node_tree

        return current_node_tree
```

3 Setting 0: Baseline Model

In this setting, we establish a baseline with an 80-10-10 split for training, validation, and testing data, respectively, and no added noise. This provides a clear view of the model's performance on clean data.

```
Experiment Setup
train_df, val_df, test_df = prepare_data(df, train_seg=80, val_seg=10, percent_noise=0)
classifier = ID3DecisionTree(impurity_metric='entropy', max_depth=None, max_impurity=0)
classifier.fit(train_df[attributes], train_df[target])
```

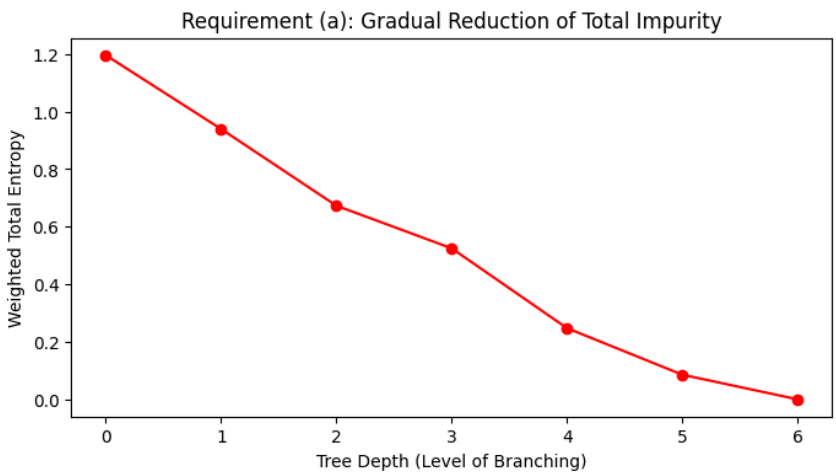

Figure 1: Impurity Reduction vs. Tree Depth in Setting 0

```
Accuracy Results
Final Training Accuracy: 100.00%
Final Validation Accuracy: 86.63%
Final Testing Accuracy: 89.66%
```

Observations: The model achieves perfect training accuracy, indicating it has fully memorized the training data. The impurity (weighted total entropy) steadily decreases as the tree depth increases, which is the expected behavior of the ID3 algorithm. The high validation and testing accuracies suggest the model generalizes well, but the gap between training and test accuracy points to some overfitting.

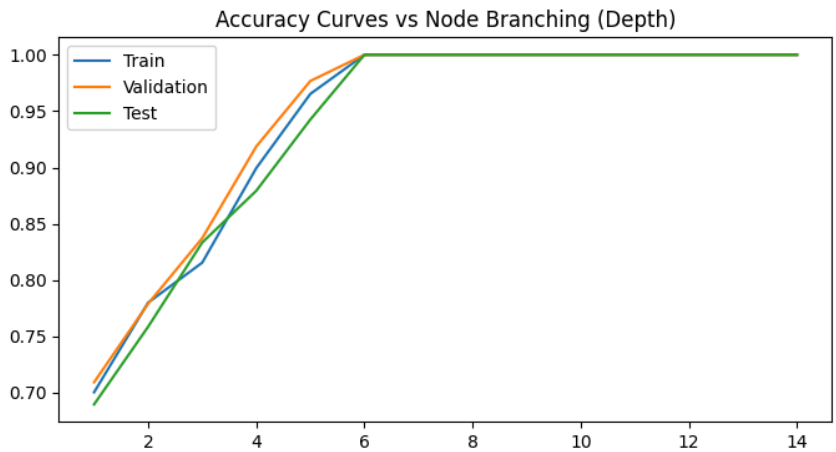

Figure 2: Accuracy Curves vs. Tree Depth in Setting 0

4 Setting 1: 60-20-20 Split

Here, the training set is reduced to 60%, while the validation and test sets are increased to 20% each. This helps to assess the model's performance with less training data and a more robust evaluation.

```
Experiment Setup
train_df, val_df, test_df = prepare_data(df, train_seg=60, val_seg=20, percent_noise=0)
classifier = ID3DecisionTree(impurity_metric='entropy', max_depth=None, max_impurity=0)
classifier.fit(train_df[attributes], train_df[target])
```

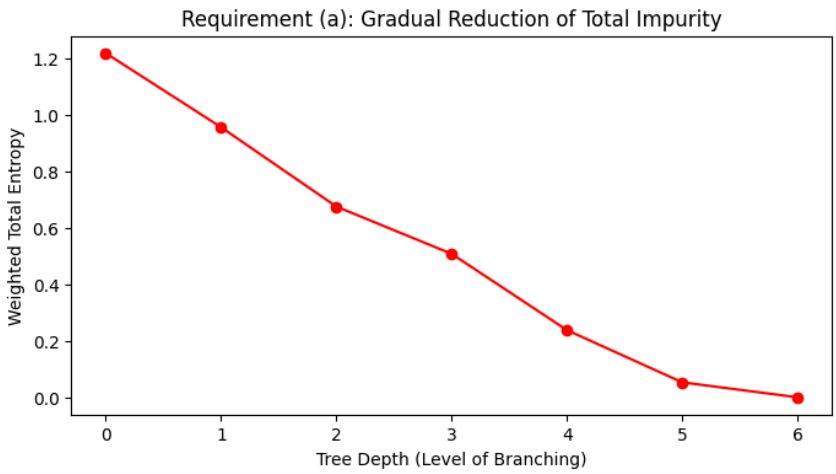

Figure 3: Impurity Reduction vs. Tree Depth in Setting 1

```
Accuracy Results
Final Training Accuracy: 100.00%
Final Validation Accuracy: 88.12%
Final Testing Accuracy: 88.76%
```

Observations: Even with a smaller training set, the model reaches 100% accuracy, showing its tendency to overfit. The validation and testing accuracies remain high and are very close to each other, which indicates that a 60% training split is still sufficient for good generalization on this particular dataset.

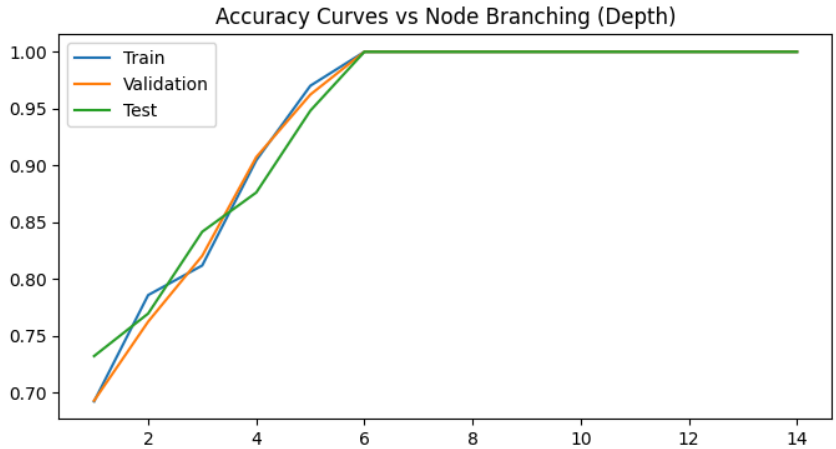

Figure 4: Accuracy Curves vs. Tree Depth in Setting 1

5 Setting 2: Introducing Noise (10%)

We revert to an 80-10-10 split but introduce 10% noise into the dataset. This tests the model's robustness to incorrect labels or attribute values.

```
Experiment Setup
train_df, val_df, test_df = prepare_data(df, train_seg=80, val_seg=10, percent_noise=10)
classifier = ID3DecisionTree(impurity_metric='entropy', max_depth=None, max_impurity=0)
classifier.fit(train_df[attributes], train_df[target])
```

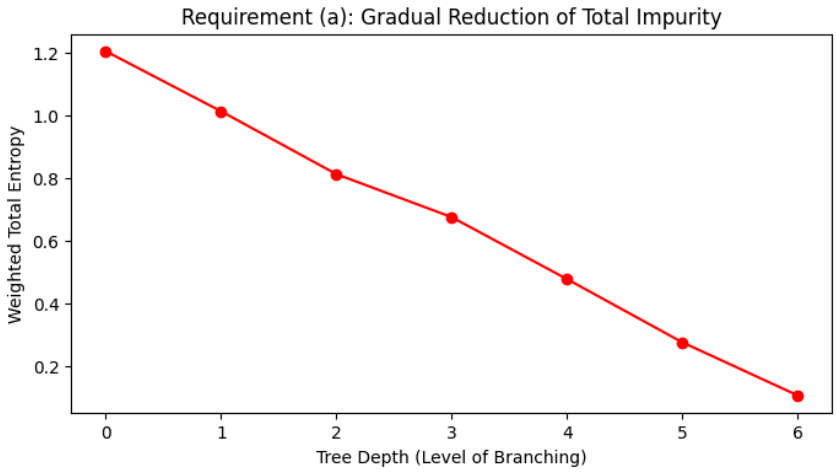

Figure 5: Impurity Reduction vs. Tree Depth in Setting 2

```
Accuracy Results
Final Training Accuracy: 94.36%
```

```
Final Validation Accuracy: 76.74%
Final Testing Accuracy: 76.44%
```

Observations: The introduction of noise prevents the model from achieving perfect training accuracy. More importantly, both validation and testing accuracies drop significantly compared to the baseline, highlighting the model's sensitivity to noisy data. The overfitting issue is more pronounced, as the gap between training and test accuracy widens.

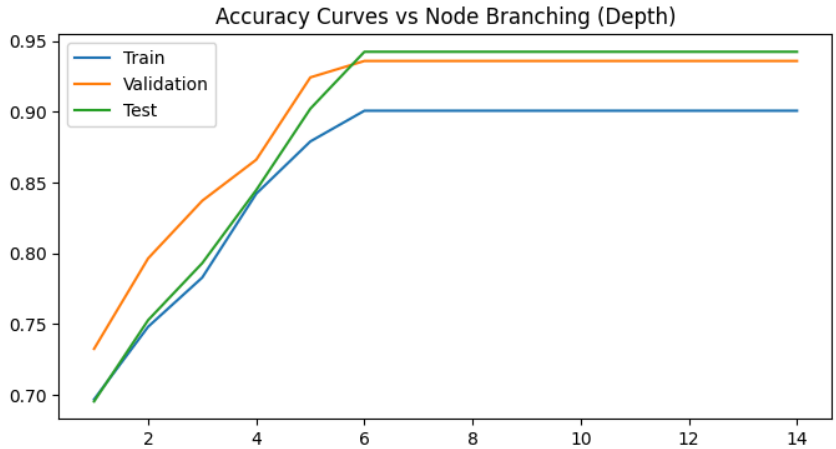

Figure 6: Accuracy Curves vs. Tree Depth in Setting 2

6 Setting 3: Noise with 60-20-20 Split

This combines the challenges of a smaller training set (60%) and 10% noise, providing a stressful test for the model's learning capability.

```
Experiment Setup
train_df, val_df, test_df = prepare_data(df, train_seg=60, val_seg=20, percent_noise=10)
classifier = ID3DecisionTree(impurity_metric='entropy', max_depth=None, max_impurity=0)
classifier.fit(train_df[attributes], train_df[target])
```

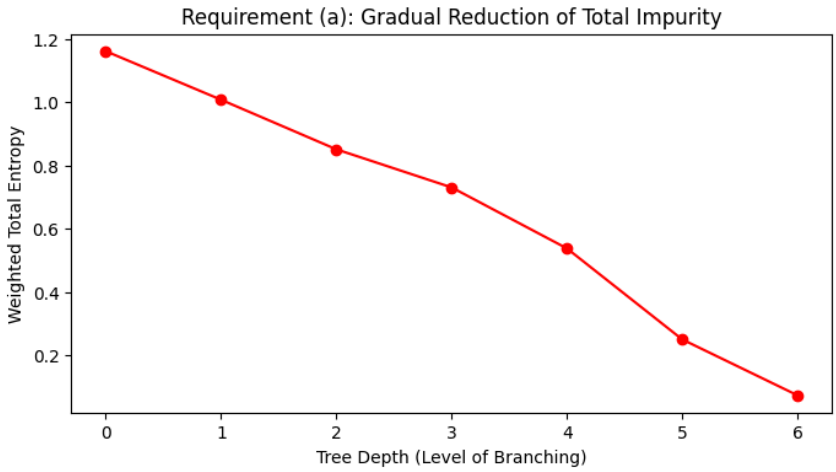

Figure 7: Impurity Reduction vs. Tree Depth in Setting 3

```
Accuracy Results
Final Training Accuracy: 94.69%
Final Validation Accuracy: 66.67%
Final Testing Accuracy: 65.42%
```

Observations: The accuracies on the validation and test sets are the lowest among all settings. The combination of less training data and noise severely hampers the model's ability to generalize. The training accuracy remains high, but the performance on unseen data is poor, which is a classic sign of a model that has learned the noise in the training data.

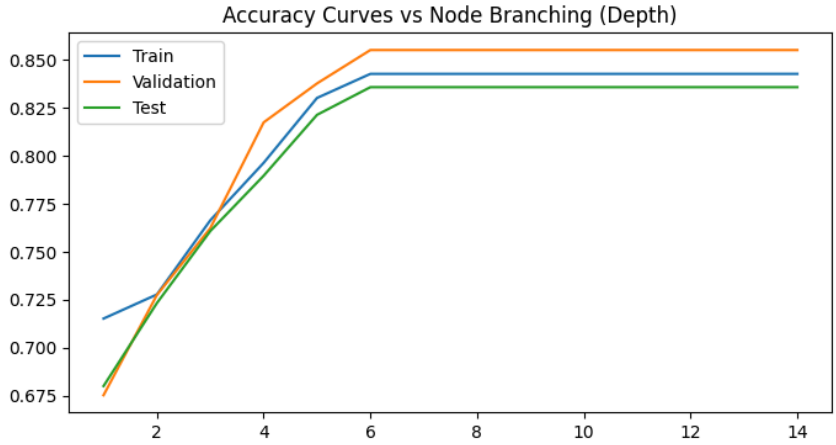

Figure 8: Accuracy Curves vs. Tree Depth in Setting 3

7 Setting 4: Pre-pruning with Max Depth and Impurity

To combat overfitting, we apply pre-pruning by setting a `max_depth` of 3 and a `max_impurity` of 0.25. The data split is 80-10-10 with no noise.

```
Experiment Setup
classifier = ID3DecisionTree(impurity_metric='entropy', max_depth=3, max_impurity=0.25)
classifier.fit(train_df[attributes], train_df[target])
```

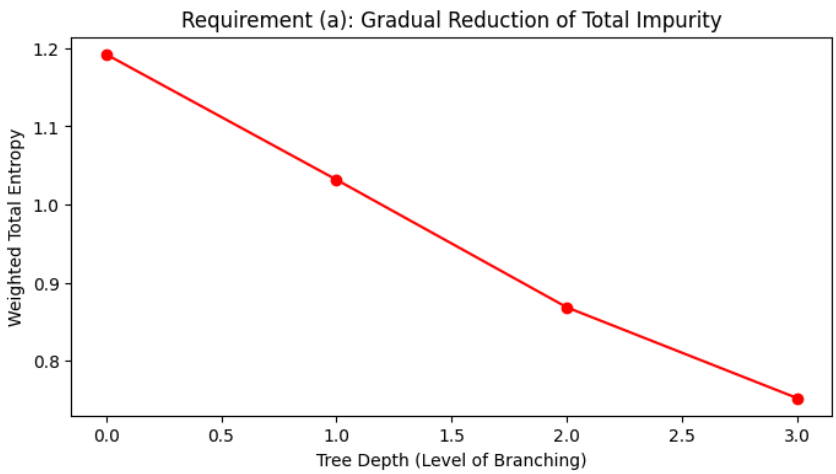

Figure 9: Impurity Reduction vs. Tree Depth in Setting 4

```
Accuracy Results
Final Training Accuracy: 76.34%
Final Validation Accuracy: 69.77%
Final Testing Accuracy: 75.29%
```

Observations: Pre-pruning successfully prevents the model from overfitting, as the training accuracy is no longer 100% and is much closer to the validation and test accuracies. While the overall accuracy is lower than the unpruned baseline, the model is much simpler and likely

more robust. The accuracy curves show that the training and test accuracies track each other more closely.

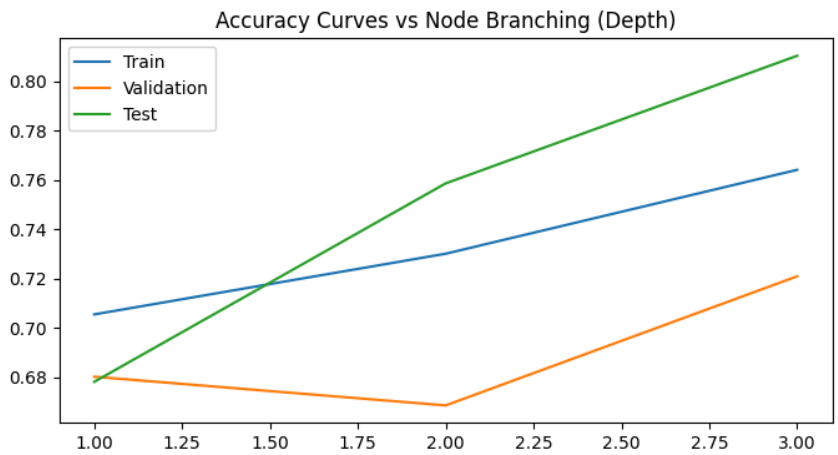

Figure 10: Accuracy Curves vs. Tree Depth in Setting 4

8 Setting 5: Using Gini Impurity

In this final setting, we switch the impurity metric from Entropy to Gini Index. The data split is 80-10-10 with no noise, allowing for a direct comparison with the baseline (Setting 0).

```
Experiment Setup
classifier = ID3DecisionTree(impurity_metric='gini', max_depth=None, max_impurity=0)
classifier.fit(train_df[attributes], train_df[target])
```

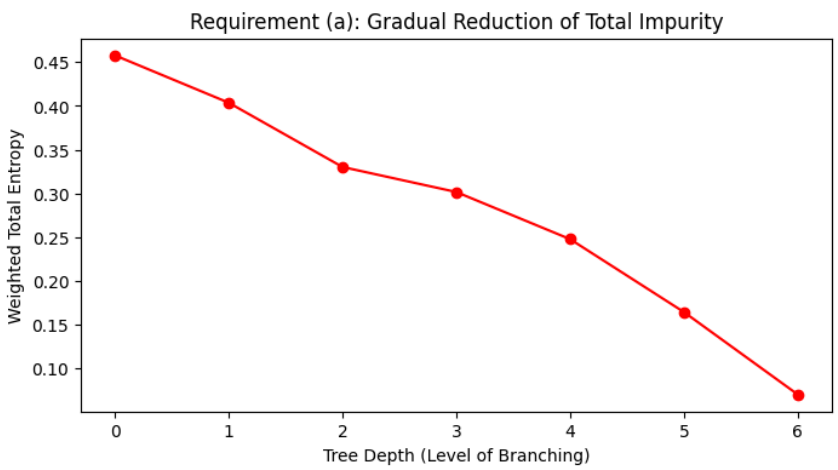

Figure 11: Impurity Reduction vs. Tree Depth in Setting 5 (Gini)

```
Accuracy Results
Final Training Accuracy: 92.47%
Final Validation Accuracy: 63.95%
Final Testing Accuracy: 67.82%
```

Observations: With Gini impurity, the model does not achieve 100% training accuracy, suggesting it might be slightly less prone to overfitting on this dataset than when using entropy. However, the validation and testing accuracies are significantly lower than the entropy-based baseline. For this specific problem, entropy appears to be a more effective impurity metric for building a generalizable tree.

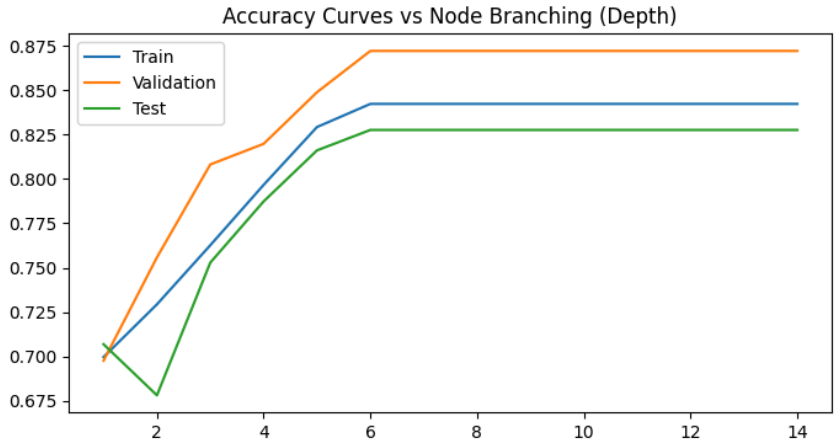

Figure 12: Accuracy Curves vs. Tree Depth in Setting 5 (Gini)

9 Conclusion

This report detailed the implementation of the ID3 decision tree algorithm and analyzed its performance under various conditions. The key takeaways are:

- The ID3 algorithm, without pruning, is highly prone to overfitting, especially on clean data, where it achieves perfect training accuracy but shows a gap when tested on unseen data.
- The model's performance is sensitive to the size of the training data, although it performed well even with a 60% split on this dataset.
- The presence of noise in the data significantly degrades the model's ability to generalize, leading to a substantial drop in validation and test accuracy.
- Pre-pruning techniques, such as setting a maximum depth and impurity threshold, are effective at reducing overfitting, resulting in a model that generalizes better at the cost of some training accuracy.
- For the Car Evaluation dataset, using Entropy as the impurity metric yielded a model with better generalization performance compared to the Gini Index.

Overall, the experiments highlight the classic trade-offs in machine learning between model complexity, bias, and variance, and demonstrate the importance of techniques like pruning and the choice of impurity metric in building robust decision tree classifiers.